UPWORK PROPOSAL ASSISTANT

Comprehensive Database System Design Document

Technical Specification Manual — Version 2.0

Project Development Group Members:

Sr. No	Group Member Name	Roll Number / ID
1	Tayyab Shahzad	NUM-BSCS-2024-77
2	Najeeba	NUM-BSCS-2024-61
3	Kashif Ali	NUM-BSCS-2024-29

Department of Computer Sciences

Namal University

Mianwali, Pakistan

Document Submission Date: 14th June, 2026

Revision History

Date	Version	Description	Approved by
21/06/2026	V 2.0	Implemented normalized 3NF schema, generated matching DDL/DML scripts with 20+ records, and added specialized admin profile structures.	Asiya Batool
15/05/2026	V 1.0	Final review completed. All conceptual sections proofread and Milestone 2 report submitted.	Asiya Batool
10/05/2026	V 0.3	Business rules drafted and EERD finalized. Relationships and cardinalities verified.	Tayyab Shahzad

Contents

List of Tables

Chapter 1

Conceptual Database Design

1.1 1.1 Entity & Data Dictionary Summary

Our system architecture rests on a clean, disjoint specialization hierarchy mapping `Person` into specialized sub-profiles: `Freelancer`, `Client`, and `Admin`. Supporting tables include `Skills`, `Proposal Template`, `Job Posting`, `Reliability Score`, and `Freelancer Analytics`. All attributes are bound by strict scalar types, primary keys, and mandatory unique index constraints to protect data state transitions across processing loops.

1.2 1.2 Data Dictionary Definitions

The physical domain attributes, storage allocations, primary keys (PK), foreign keys (FK), and structural constraint configurations for all 12 relations in the Upwork Proposal Assistant database schema are systematically cataloged in the tables below. To provide a thorough normalization perspective, each architectural configuration is accompanied by an expanded technical analysis detailing its exact structural dependencies, primary key selection rationale, and cross-table operational enforcement mechanisms:

1.2.1 1.2.1 1. Person Table

The `Person` table acts as the system's core foundational supertype relation within our entity specialization layout. It consolidates demographic and identity parameters common to every user on the platform, preventing redundant text allocations across downstream profiles. By decoupling core strings (such as emails and portfolio URLs) into this single table, the schema naturally guarantees First Normal Form (1NF) atomic string compliance and rules out multi-point data modification issues during registration.

Table 1.1: Person Table Schema Definition

Attribute Name	**Data Type**	**Key**	**Null/Unique**	**Description / Constraints**
`person_id`	INT	PK	NOT NULL	System-managed, auto-incremented tracking token. Enforces an absolute system-wide identity standard for entity clustering.
`f_name`	VARCHAR(50)	—	NOT NULL	User's legal first name. Enforced via clean regex validation to filter out numerical values or specialized symbol strings.
`Lname`	VARCHAR(50)	—	NOT NULL	User's legal last name. Implements strict entry policies ensuring that non-blank values exist for profile rendering components.
`email`	VARCHAR(100)	—	UNIQUE	Primary electronic communication address. Evaluated via transactional trigger checking standard RFC syntax (`char@domain.ext`).
`country`	VARCHAR(50)	—	NULL	Geographic registration country location. Tied directly to a master country list to streamline geographical platform metrics.
`upwork_profile_url`	VARCHAR(255)	—	UNIQUE	Validated web link referencing the external verified Upwork account profile. Prevents cross-account duplicate manipulation.

1.2.2 1.2.2 2. Freelancer Table

The `Freelancer` entity outlines a strict specialized subtype partition branched directly from the core `Person` index. Its primary key (`freelancer_id`) is explicitly mapped as a foreign key referencing `Person(person_id)` under a `ON DELETE CASCADE` policy. This 1:1 specialization layout removes non-prime operational variables—such as specific hourly pricing boundaries and industry tenure lengths—away from general system profiles, meeting 3NF requirements since all fields depend exclusively on the candidate key.

Table 1.2: Freelancer Table Schema Definition

Attribute Name	Data Type	Key	Null/Unique	Description / Constraints
freelancer_id	INT	PK, FK	NOT NULL	Relational link to Person(person_id). Operates with a cascade policy on deleting parent records to maintain platform consistency.
password	VARCHAR(255)	—	NOT NULL	Secure security storage token. Holds highly complex cryptographic strings hashed via standard Argon2id salt hashing parameters.
hourly_rate	DECIMAL(6,2)	—	NOT NULL	Assigned operational service cost per hour. Governed by a validation boundary constraint ensuring values remain above $0.00.
experience_years	INT	—	NULL	Tracked total industry tenure length. Bound check logic maintains numerical tracking between 0 and 60 to filter outlier records.

1.2.3 1.2.3 3. Client Table

The Client relation records operational attributes specific to enterprise or independent employers posting project briefs. To prevent database design errors, client metrics (such as aggregate hiring ratios and qualitative textual feedback historical logs) are strictly kept out of global tracking sheets. The table's primary key serves as a direct referential anchor to the master Person table, ensuring clean relational boundaries where business metrics do not interfere with developer profiles.

1.2.4 1.2.4 4. Admin Table

The Admin framework manages security clearances, operational states, and governance privileges across platform elements. Keeping administrative rows in an isolated subtype structure means internal system parameters (like moderator assignments and duty states) remain securely separated from public-facing user configurations. This approach simplifies access controls and prevents structural or security oversights.

Table 1.3: Client Table Schema Definition

Attribute Name	Data Type	Key	Null/Unique	Description / Constraints
`client_id`	INT	PK, FK	NOT NULL	Subtype anchor matching `Person(person_id)`. Supports complete delete cascading loops across enterprise profile sets.
`company_name`	VARCHAR(100)	—	NULL	Corporate or registered business title. Stored as free-text formatting allowing for specific industrial enterprise extensions.
`success_rate`	DECIMAL(5,2)	—	NULL	Monitored historical client completion standard. Automatically calculated via an isolated procedural system loop from 0.00 to 100.00.
`feedback`	TEXT	—	NULL	Long-form text field capturing aggregated freelancer feedback, reviews, and qualitative platform notes.

1.2.5 1.2.5 5. Skills Table

The `Skills` table acts as a master lookup directory that catalogs technical competencies on the platform. It enforces a strict unique index constraint on the `skill_name` attribute string to prevent spelling variations or duplicate entries from corrupting the lookup index. This layout guarantees a highly normalized repository for clean data tracking.

1.2.6 1.2.6 6. Freelancer Skill Map Table

This associative entity defines structural mapping configurations between users and technical competencies, cleanly resolving the complex many-to-many relationship linking freelancers to multiple skills. This intersection mapping pattern avoids storage anomalies and array violations by recording dependencies strictly as individual binary key-pair rows, fulfilling 2NF requirements.

Table 1.4: Admin Table Schema Definition

Attribute Name	Data Type	Key	Null/Unique	Description / Constraints
admin_id	INT	PK, FK	NOT NULL	Relational link to Person(person_id). Separates core infrastructure governance from typical consumer activities.
role	VARCHAR(50)	—	NOT NULL	Explicit operational group classification. Constrained via specific configuration boundaries: 'SuperAdmin', 'Moderator', 'Analyst'.
joined_date	DATE	—	NOT NULL	Calendar date marking administrative assignment. System records enforce formatting matching the standard YYYY-MM-DD pattern.
status	VARCHAR(20)	—	NOT NULL	Tracks current administrative availability. Enforced state checks include values: 'Active', 'Suspended', 'On-Leave'.

Table 1.5: Skills Table Schema Definition

Attribute Name	Data Type	Key	Null/Unique	Description / Constraints
skill_id	INT	PK	NOT NULL	Auto-generated unique tracking integer. Acts as the static master cluster anchor across associative map references.
skill_name	VARCHAR(100)	—	UNIQUE	Normalized text name of technical competency (e.g., 'MySQL', 'Python'). Enforces case-insensitive filtering mechanics.
skill_level	VARCHAR(30)	—	NOT NULL	Intended capability tier tracking. Validated exclusively against a localized string set: 'Beginner', 'Intermediate', 'Expert'.

1.2.7 1.2.7 7. Job Posting Table

The Job_Posting table catalogs structural details for individual contract listings uploaded by verified employers. The relation adheres completely to 3NF standards because every single descriptive non-prime variable—including budget limits, funding structures, and experience criteria checks—depends entirely on the primary surrogate identifier job_id.

Table 1.6: Freelancer Skill Map Table Schema Definition

Attribute Name	Data Type	Key	Null/Unique	Description / Constraints
`freelancer_id`	INT	PK, FK	NOT NULL	Part of composite primary key. Cascades row changes if the underlying freelancer user is modified or dropped.
`skill_id`	INT	PK, FK	NOT NULL	Part of composite primary key. Implements restrictive foreign key blocks to preserve valid tracking logs when skills are in active use.

This prevents transitive dependencies across employer entities.

1.2.8 1.2.8 8. Job Skill Map Table

The `Job_Skill_Map` associative relation links specific technical competencies to active job postings. Breaking down many-to-many connections into simple key-pair assignments ensures that when required skills are modified, the operations remain localized, fast, and completely free of unintended data mutations.

1.2.9 1.2.9 9. Proposal Template Table

The `Proposal_Template` repository tracks standard template configurations crafted by administrative moderators. Isolating layout components from the main application log sheets optimizes storage and ensures 3NF compliance. User profiles can reuse these boilerplates simply by referencing the unique `template_id` key, keeping overall data overhead low.

Table 1.7: Job Posting Table Schema Definition

Attribute Name	Data Type	Key	Null/Unique	Description / Constraints
job_id	INT	PK	NOT NULL	Auto-incremented primary tracking surrogate key. Uniquely logs each marketplace listing.
client_id	INT	FK	NOT NULL	References Client(client_id). Restricts job record deletion if an active client account continues to run project operations.
job_title	VARCHAR(150)	—	NOT NULL	Title label identifying the contract listing. Cleaned via character rules to remove excessive punctuation.
job_description	TEXT	—	NOT NULL	Detailed long-form text cataloging the technical scope, milestones, and explicit deliverables required for the project.
experience_level	VARCHAR(20)	—	NOT NULL	Targeted talent requirement profile. Bound check filters parameters exclusively to: 'Entry', 'Mid', 'Expert'.
job_type	VARCHAR(20)	—	NOT NULL	Financial structural designation. System constraint limits rows strictly to 'Fixed-price' or 'Hourly' billing methods.
project_duration	VARCHAR(50)	—	NULL	Expected timeframe window length for contract delivery (e.g., '3 to 6 months'). Stored for frontend filtering options.
budget_amount	DECIMAL(10,2)	—	NOT NULL	Maximum allocated project funding. Constrained to enforce values greater than $0.00 to block bad test entries.
posted_date	TIMESTAMP	—	NOT NULL	Automatic database transaction record marker. Captures the precise server epoch time when the record transitions to active status.
proposal_count	INT	—	DEFAULT 0	Live application transaction log tally. Automatically tracked and incremented via proposal generation loops.

Table 1.8: Job Skill Map Table Schema Definition

Attribute Name	Data Type	Key	Null/Unique	Description / Constraints
job_id	INT	PK, FK	NOT NULL	Part of composite primary key sequence. Cascades directly with the master Job_Posting records to eliminate orphan data dependencies.
skill_id	INT	PK, FK	NOT NULL	Part of composite primary key sequence. Enforces validation checks ensuring that linked lookups point to real, verified skills.

Table 1.9: Proposal Template Table Schema Definition

Attribute Name	Data Type	Key	Null/Unique	Description / Constraints
template_id	INT	PK	NOT NULL	Auto-generated unique key indexing specific template items. Maps structural boilerplate configurations cleanly.
admin_id	INT	FK	NULL	References Admin(admin_id). Implements a set-null configuration if the tracking admin record is purged from the platform.
template_name	VARCHAR(100)	—	NOT NULL	Core name identifying the specific system cover letter boilerplate (e.g., 'React Architecture Basic').
category	VARCHAR(50)	—	NOT NULL	Classification tag used to bundle similar items together (e.g., 'Web Development', 'Data Engineering').
template_content	TEXT	—	NOT NULL	Master boilerplate body layout text block. Features specific token anchors utilized during dynamic variable mapping stages.
usage_count	INT	—	DEFAULT 0	Analytical counter tracking total freelancer usage. Monitored via a trigger routine to assess asset utility.
success_rate	DECIMAL(5,2)	—	NULL	Success conversion performance measurement. Tracks proposal template items that transition into winning project hires.

1.2.10 1.2.10 10. Reliability Score Table

The `Reliability_Score` entity logs client metric snapshots for real-time risk profiling. Keeping automated calculations outside of primary profiling tables ensures high 3NF performance. This isolation guarantees that background analytical score adjustments do not inadvertently lock or corrupt primary user records.

Table 1.10: Reliability Score Table Schema Definition

Attribute Name	**Data Type**	**Key**	**Null/Unique**	**Description / Constraints**
`score_id`	INT	PK	NOT NULL	Primary system tracking index. Isolates historical reliability runs away from primary business transaction entities.
`client_id`	INT	FK	NOT NULL	References `Client(client_id)`. Implements a cascade framework to ensure analytical row cleanups trigger when clients leave.
`score`	DECIMAL(5,2)	—	NOT NULL	Calculated system reliability metric. Constrained strictly to values between 0.00 and 100.00 to avoid computational anomalies.
`risk_level`	VARCHAR(20)	—	NOT NULL	Categorized evaluation band. Enforced via database bounds tracking precisely: 'Low', 'Medium', 'High'.
`recommendation_level`	VARCHAR(50)	—	NOT NULL	System behavioral assessment flag. Outputs values like 'Highly Recommended' or 'Avoid' to inform user workflows.
`calculated_date`	TIMESTAMP	—	NOT NULL	Precise date and time tracking when the procedural scoring update loop completed execution.

1.2.11 1.2.11 11. Freelancer Analytics Table

The `Freelancer_Analytics` table maintains execution summaries of user conversions. Storing these dynamic performance stats apart from core profiles helps avoid structural

anomalies. This pattern ensures that rapid analytical tracking updates can run smoothly without risking data locks on authentication rows.

Table 1.11: Freelancer Analytics Table Schema Definition

Attribute Name	Data Type	Key	Null/Unique	Description / Constraints
analytics_id	INT	PK	NOT NULL	System tracking key assigned to isolate rolling analytical computation structures from the core application records.
freelancer_id	INT	FK	NOT NULL	References Freelancer(freelancer_id). Employs a unique mapping constraint to limit tracking arrays to a single row per freelancer.
total_proposals	INT	—	DEFAULT 0	Rolling count of all submitted contracts. Maintained via localized database transactional insert listeners.
accepted_proposals	INT	—	DEFAULT 0	Historical record tally documenting positive hire responses. Increments automatically through specific status triggers.
rejected_proposals	INT	—	DEFAULT 0	Total tally tracking unapproved applications or passed contract attempts across the freelancer's account history.
success_rate	DECIMAL(5,2)	—	DEFAULT 0.00	Computed percentage ratio tracking overall conversion performance. Restricted between 0.00 and 100.00.

1.2.12 1.2.12 12. Generated Proposals Table

The Generated_Proposals table records transactional data from cover letter assembly loops. It coordinates foreign keys from four distinct parent entities into a unified structure. Because all final cover letter content blocks and status flags depend entirely on the unique primary key proposal_id, the table completely avoids transitive anomalies and satisfies all 3NF boundaries.

Table 1.12: Generated Proposals Table Schema Definition

Attribute Name	Data Type	Key	Null/Unique	Description / Constraints
proposal_id	INT	PK	NOT NULL	Unique auto-increment tracking key capturing individual proposal creation runs across the platform.
freelancer_id	INT	FK	NOT NULL	References Freelancer(freelancer_id). Retains tracking logs using a restricted deletion structure to prevent operational data loss.
client_id	INT	FK	NOT NULL	References Client(client_id). Establishes clear relational reporting pathways mapping proposals directly back to original job publishers.
job_id	INT	FK	NOT NULL	References Job_Posting(job_id). Links the submission row to specific marketplace listings, cascading seamlessly if the listing is cleared.
template_id	INT	FK	NOT NULL	References Proposal_Template(template_id). Maps the base layout style used to compile the cover letter body text.
proposal_content	TEXT	—	NOT NULL	The final customized cover letter text block generated by the platform algorithm, tailormade to fit the project scope.
generated_date	TIMESTAMP	—	NOT NULL	Record creation time. Automatically logs when the platform completes compilation of the proposal content.
proposal_status	VARCHAR(30)	—	NOT NULL	Tracks current submission status. Enforced boundaries include strings: 'Pending', 'Submitted', 'Accepted', 'Rejected'.
submission_date	TIMESTAMP	—	NULL	Captured timestamp logging exactly when the proposal was delivered to the external API endpoint. Remains null during drafting phases.

Chapter 2

Logical Database Design

2.1 2.1 Relational Schema Mapping Rules

The database tables are structured based on Third Normal Form (3NF) relational criteria to achieve high structural stability. The blueprint maps the conceptual specialization model into concrete physical relations using a specialized table-per-child hierarchy strategy. Under this arrangement, the parent entity `Person` maintains identity tokens common to all actors on the platform. Sub-profiles extend these parent definitions.

Rather than duplicating common descriptors across multiple endpoints, child relations maintain strict referencing integrity by reusing the parent's identifier as their primary key while binding it concurrently under an index-validated foreign key constraint. Associative intersection bridges decouple many-to-many linkages into explicit key-pairs, ensuring that multi-valued attributes remain atomic.

2.2 2.2 Functional Dependencies & Normalization Analysis

All data fields map directly and explicitly to primary composite candidate keys. The structural layout of all 12 operational relations is thoroughly verified to meet the performance bounds of Third Normal Form (3NF).

1. **First Normal Form (1NF):** Every structural item is strictly atomic. Text and description frameworks avoid packed arrays by isolating repeating sequences.
2. **Second Normal Form (2NF):** All transactional schemas possess full functional dependency on their primary tracking elements. In relations with composite primary configurations, such as mapping bridges, partial dependencies are completely absent.

3. **Third Normal Form (3NF):** Transitive functional dependencies are completely eliminated. Descriptive text fields depend directly on the primary key identifier, avoiding indirect functional dependencies on secondary variables.

Chapter 3

Implementation

3.1 3.1 Database Engine and Environment Settings

The data infrastructure runs concurrently under a local instance of the MySQL 8.0 Engine. The underlying platform maps physical data sectors via the InnoDB transaction processing engine. This setup handles strict enforcement of database ACID properties (Atomicity, Consistency, Isolation, Durability) alongside native row-level write-locking mechanics during active query loads.

3.2 3.2 Database Programmability Objects

The programmable logic layer implements backend control mechanisms via stored routines and data tracking automation rules. Table 3.1 outlines these functional behaviors:

3.3 3.3 Interface Payload Mapping

The system layout registers connection paths securely handling transactional payloads smoothly. This section serves as a descriptive blueprint detailing how the platform's backend relational schema links with external operational frontends. The core purpose of the data schemas defined in Chapter 1 is to seamlessly pass data payloads between the database instances and client interfaces without structural formatting bottlenecks or string escape failures.

By standardizing primary identity references across individual staging arrays, the data catalog establishes a structured map that application connectors can query instantly. This foundational layout ensures that when subsequent development phases introduce runtime interface integrations, the pre-built table schemas are fully prepared to support stable, continuous system querying.

Table 3.1: Database Programmability Objects Directory

Sr.	Object Name	Object Type	Description	Input / Output Parameters
01	Calculate_Reliability	Stored Procedure	Evaluates client success rates against policy data, updates risks, and appends a score row.	**Input:** `c_id` INT, `raw_score` DECIMAL **Output:** Inserts to `Reliability_Score`.
02	Increment_Template_Usage	Trigger	Automatically increments the template usage counter when a new proposal row is added.	**Event:** AFTER INSERT on `Generated_Proposals` table.
03	Sync_Freelancer_Metrics	Stored Procedure	Automatically recalculates totals, acceptance counters, and success rate values for a freelancer.	**Input:** `f_id` INT **Output:** Modifies row inside `Freelancer_Analytics`.

Bibliography

[1] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Pearson, 2016.

[2] MySQL Reference Manual, "Stored Procedures, Functions, and Triggers Infrastructure," Oracle Corp, 2026.

Appendix: AI Usage and Prompts

In compliance with project design evaluation guidelines, the development prompts used with generative artificial intelligence models are documented below:

- **Prompt 1 (Schema Refactoring):** "Review my Upwork database design layout attributes. Fix any potential decimal data type truncation bottlenecks within the job posting financial budgets and client reliability performance score range values to ensure it safely handles up to 100% metric values without runtime crashes."
- **Prompt 2 (Transactional Scaling):** "Generate a fully expanded, interconnected relational seed script mapping out 2 distinct administrative roles (one for system user metrics analysis, one for outgoing template tracking) along with more than 20 rows of realistic transactional mock records per operational table to test structural performance."
- **Prompt 3 (Documentation Mapping):** "Map my updated relational set structures into a comprehensive logical validation report structured under formal Chapter 3 and Chapter 4 sections following Namal University project templates."